

**PONTIFICIA UNIVERSIDAD
CATÓLICA DEL PERÚ**

FACULTAD DE CIENCIAS E INGENIERÍA



ISA - Informe de Selección de Algoritmos

Proyecto de Diseño y Desarrollo de Software

EQUIPO:

6F

INTEGRANTES:

Renzo Kei Shimabukuro Kohatsu - 20210477

Angel Freddy Cerdán Cigüeñas - 20220749

Álvaro Pesantes Advíncula - 20222928

Pablo Elias Atahua Ramirez - 20195867

PROFESOR:

Prof. Fernández Sánchez, Juan Carlos

HORARIO:

0983

Lima, abril de 2026

HISTORIAL DE REVISIONES

Fecha	Versión	Descripción	Autor
07/04/2026	1.0	Identificación y caracterización de algoritmos metaheurísticos para el componente planificador. Se seleccionan y justifican dos algoritmos candidatos con base en la literatura científica.	Equipo 6F
13/04/2026	2.0	Se agregó partes relevantes del código y pseudocódigo	Equipo 6F
22/04/2026	3.0	Correcciones menores hechas por el JP	Equipo 6F

1. Introducción

El presente informe tiene como propósito identificar, analizar y justificar los algoritmos metaheurísticos seleccionados para el componente planificador del sistema de gestión de traslado de equipajes de la empresa Tasf.B2B. El componente planificador es el núcleo del sistema: debe calcular rutas óptimas para el traslado de maletas entre aeropuertos de América, Asia y Europa, garantizando el cumplimiento de plazos comprometidos (máximo un día para el mismo continente y dos días para continentes distintos), respetando las restricciones de capacidad de vuelos y almacenes, y reaccionando ante cancelaciones de vuelos con replanificación ágil.

Por la naturaleza combinatoria y multirestricción del problema equivalente a un problema de enrutamiento con restricciones de capacidad (CVRP) extendido sobre una red de transporte aéreo se optó por algoritmos metaheurísticos, los cuales ofrecen soluciones de alta calidad en tiempos computacionales razonables sin necesidad de explorar exhaustivamente el espacio de búsqueda.

2.Descripción del Problema de Optimización

El problema a resolver puede formularse como la asignación óptima de rutas de traslado para lotes de maletas sobre una red de vuelos con las siguientes características y restricciones:

Elemento	Descripción
Red de nodos	Aeropuertos de América, Asia y Europa. Capacidad de almacén: 500-800 maletas por ciudad.
Arcos / Vuelos	Vuelos entre pares de aeropuertos. Capacidad: 150-250 maletas (mismo continente) y 150-400 maletas (distinto continente). Frecuencia: ≥ 1 vuelo/día.
Tiempo de traslado	0,5 días entre ciudades del mismo continente; 1 día entre ciudades de distintos continentes.
Plazos de entrega	Máximo 1 día (mismo continente) y máximo 2 días (distinto continente).

Contingencias	Cancelaciones de vuelos que requieren replanificación automática sin reiniciar el proceso.
Escenarios	Día a día (tiempo real), simulación de periodo (3, 5 días o semanal) y simulación hasta el colapso.

La función objetivo principal es minimizar el riesgo de incumplimiento de los plazos de entrega, combinando tiempo total de ruta, saturación de nodos intermedios y número de escalas, sujeto a las restricciones de capacidad de vuelos y almacenes. Este problema pertenece a la clase NP-hard, lo que hace inviable la búsqueda exhaustiva para redes de escala real y justifica el uso de metaheurísticas.

3. Algoritmo 1: Colonia de Hormigas (ACO)

3.1 Descripción del algoritmo

El Algoritmo de Colonia de Hormigas (Ant Colony Optimization, ACO) es una metaheurística de inteligencia de enjambre propuesta por Marco Dorigo en 1992, formalizada posteriormente en Dorigo & Gambardella (1997) con la variante Ant Colony System (ACS). Su inspiración biológica proviene del comportamiento de las hormigas reales: al desplazarse, depositan una sustancia química denominada feromona sobre los caminos recorridos, lo que orienta a las siguientes hormigas hacia las rutas más prometedoras. Con el tiempo, las rutas subóptimas pierden atractivo por la evaporación de la feromona, mientras que las rutas de mejor calidad concentran mayor densidad de feromona y son seleccionadas con mayor probabilidad. Formalmente, el algoritmo mantiene una matriz de feromonas $\tau(i, j)$ sobre los arcos del grafo de búsqueda. En cada iteración, una colonia de “m” hormigas construye soluciones completas de forma probabilística, guiándose tanto por la feromona acumulada como por información heurística $\eta(i, j)$ —típicamente el inverso del costo del arco. La probabilidad de que la hormiga k se desplace del nodo i al nodo j se calcula como:

$$P(i \rightarrow j) = [\tau(i, j)^\alpha \cdot \eta(i, j)^\beta] / \sum_l [\tau(i, l)^\alpha \cdot \eta(i, l)^\beta]$$

donde α controla el peso de la feromona y β el peso de la información heurística. Al finalizar la construcción, las feromonas se actualizan reforzando los arcos de las mejores soluciones y aplicando una tasa de evaporación ρ para evitar estancamiento prematuro.

3.2 Modelado del Problema de Tsf.B2B

El problema de planificación de rutas de Tsf.B2B puede representarse como un grafo dirigido $G = (V, A)$, donde:

- V = conjunto de aeropuertos de la red (nodos), cada uno con un atributo de capacidad de almacén [500-800 maletas].
- A = conjunto de rutas de vuelo disponibles entre pares de aeropuertos (arcos), cada una con atributos de: capacidad de transporte [150-250 maletas mismo continente; 150-400 intercontinental], tiempo de traslado [0.5 días mismo continente, 1 día distinto continente] y frecuencia de vuelos por día.

Cada hormiga representa un agente planificador que construye una ruta completa para un lote de maletas desde su aeropuerto de origen hasta el de destino, respetando las restricciones de capacidad de vuelo, capacidad de almacén en nodos intermedios y el plazo máximo de entrega comprometido.

3.3 Función Heurística

La información heurística $\eta(i, j)$ orienta a las hormigas hacia arcos convenientes para el negocio. Se propone una función heurística compuesta:

$$\eta(i, j) = w_1 \cdot (1/\text{tiempo}(i, j)) + w_2 \cdot (\text{cap_disp_vuelo}/\text{cap_max}) \\ + w_3 \cdot (\text{almacén_disp}(j)/\text{cap_max_almacén})$$

donde w_1 , w_2 y w_3 son pesos configurables como parámetros del sistema. Esta función penaliza arcos con tiempos de traslado altos, vuelos con poca capacidad remanente y aeropuertos con almacenes próximos a saturación, alineándose con el objetivo de minimizar el riesgo de incumplimiento (RAN-01) y evitar el colapso de la red.

3.4 Actualización de Feromonas y Criterio de Refuerzo

Tras evaluar todas las soluciones de una iteración, la actualización de feromonas seguirá la estrategia global-best de ACS:

$$\tau(i, j) \leftarrow (1 - \rho) \cdot \tau(i, j) + \rho \cdot \Delta\tau(i, j)$$

donde $\Delta\tau(i, j) = Q / f(S^*)$ si el arco (i, j) pertenece a la mejor solución S^* encontrada, y 0 en caso contrario. La función objetivo $f(S)$ a minimizar combina tiempo total de ruta, riesgo de saturación de nodos intermedios y número de escalas, ponderados según las prioridades operativas.

3.5 Adaptación para Replanificación ante Cancelaciones

Ante la cancelación de un vuelo, el arco afectado puede ser temporalmente penalizado reduciendo su feromona a τ_{min} , lo que redirige automáticamente a las hormigas hacia rutas alternativas en la siguiente iteración. Esto permite replanificar de forma reactiva sin reiniciar el proceso desde cero.

3.6 Justificación de Selección

ACO es especialmente adecuado para el problema de Tasf.B2B por las siguientes razones:

- **Naturaleza del problema:** Es un problema de enrutamiento con restricciones múltiples sobre un grafo, exactamente el tipo de problema para el que ACO fue diseñado y validado extensamente en la literatura (Bell & McMullen, 2004; Dorigo & Gambardella, 1997).
- **Múltiples restricciones simultáneas:** La información heurística compuesta permite incorporar capacidad de vuelo, capacidad de almacén y plazo de entrega dentro de una única función guía, sin necesidad de transformaciones complejas de la función objetivo.
- **Adaptabilidad dinámica:** La estructura de feromonas permite responder a cancelaciones de vuelos degradando arcos afectados, lo que es esencial en un entorno operativo con interrupciones frecuentes.
- **Escalabilidad:** ACO ha demostrado buen desempeño en redes de gran escala (Dorigo et al., 2006), compatible con la red de aeropuertos en América, Asia y Europa que maneja Tasf.B2B.
- **Comparabilidad experimental:** Al ser un algoritmo con parámetros claramente definidos (α , β , ρ , número de hormigas, iteraciones máximas), facilita la experimentación numérica comparativa requerida en el objetivo O6 del proyecto.

3.7 Referencias Bibliográficas

- Dorigo, M., & Gambardella, L. M. (1997). Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1), 53–66. <https://doi.org/10.1109/4235.585892> (IEEE)
- Stützle, T., & Hoos, H. H. (2000). MAX–MIN ant system. *Future Generation Computer Systems*, 16(9), 889–914. [https://doi.org/10.1016/S0167-739X\(00\)00043-1](https://doi.org/10.1016/S0167-739X(00)00043-1) (Scopus)

- Bell, J. E., & McMullen, P. R. (2004). Ant colony optimization techniques for the vehicle routing problem. *Advanced Engineering Informatics*, 18(1), 41–48. <https://doi.org/10.1016/j.aei.2004.07.001> (Scopus)
- Dorigo, M., Birattari, M., & Stutzle, T. (2006). Ant colony optimization. *IEEE Computational Intelligence Magazine*, 1(4), 28–39. <https://doi.org/10.1109/MCI.2006.329691> (IEEE)
- Hu, X.-M., Zhang, J., Chung, H. S.-H., Li, Y., & Liu, O. (2010). SamACO: Variable sampling ant colony optimization algorithm for continuous optimization. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 40(6), 1555–1566. <https://doi.org/10.1109/TSMCB.2010.2048041> (IEEE)
- Gao, W., Zhang, R., & Xu, M. (2025). A GPR-ACO hybrid algorithm: solving the Airport Baggage Transport Vehicles Scheduling problem. En 2025 Youth Academic Annual Conference of Chinese Association of Automation (YAC). IEEE. <https://doi.org/10.1109/YAC66630.2025.11150246> (IEEE)

3.8 Pseudocódigo

A continuación se presenta el pseudocódigo formal del algoritmo Ant Colony System (ACS).

- ALGORITMO PRINCIPAL: ACS Planner

ENTRADA: red $G = (V, A)$, lista de envíos S , configuración C
 SALIDA: PlannerResult con rutas óptimas y métricas

INICIO

// Inicialización

$\tau_0 \leftarrow 1 / (\text{numAnts} \cdot f(\text{solución_greedy}))$

PARA cada arco (i, j) en A :

$\tau(i, j) \leftarrow \tau_0$

FIN PARA

$S^* \leftarrow \text{null}$

$f(S^*) \leftarrow \infty$

// Ciclo principal

PARA $\text{iter} = 1$ HASTA maxIter :

// Construcción de soluciones

```

PARA cada hormiga k = 1 HASTA numAnts:
    PARA cada envío s en S:
        ruta_k[s] ← ConstruirRuta(s.origen, s.destino, G,  $\tau$ , C)
    FIN PARA
    f_k ← EvaluarSolución(ruta_k)

    SI f_k < f(S*):
        S* ← ruta_k
        f(S*) ← f_k
    FIN SI
FIN PARA

// Actualización global-best de feromonas
ActualizarFeromonas( $\tau$ , S*, C. $\rho$ , C.Q)

Registrar en log: iter, f(S*), rutas de S*
FIN PARA

RETORNAR PlannerResult(S*, f(S*), iter)
FIN

```

- FUNCIÓN: ConstruirRuta

ENTRADA: origen, destino, red G, matriz τ , configuración C
 SALIDA: Route con lista de vuelos y tiempo total

```

INICIO
    nodo_actual ← origen
    ruta ← []
    tiempo_acumulado ← 0
    visited ← {origen}

    MIENTRAS nodo_actual ≠ destino:

        vecinos ← ObtenerVuelosDisponibles(nodo_actual, G)
        vecinos ← Filtrar(vecinos, capacidad > 0, no cancelado)

        SI vecinos está vacío:
            LANZAR excepción "Sin ruta disponible"
        FIN SI

        j ← SeleccionarSiguienteNodo(nodo_actual, vecinos,  $\tau$ , C)

        vuelo ← ObtenerMejorVuelo(nodo_actual, j)
        ruta.agregar(vuelo)
        tiempo_acumulado ← tiempo_acumulado + vuelo.tiempoTraslado
        visited.agregar(j)
        nodo_actual ← j
    FIN MIENTRAS

```



```

    RETORNAR Route(ruta, tiempo_acumulado)
FIN

```

- FUNCIÓN: SeleccionarSiguienteNodo

ENTRADA: nodo i , vecinos disponibles, matriz τ , configuración C
 SALIDA: nodo j seleccionado

```

INICIO
    // Calcular  $\eta(i,j)$  para cada vecino
    PARA cada vecino  $j$ :
         $\eta(i,j) \leftarrow C.w1 \cdot (1/\text{tiempo}(i,j))$ 
            +  $C.w2 \cdot (\text{cap\_disp\_vuelo}(i,j) / \text{cap\_max\_vuelo})$ 
            +  $C.w3 \cdot (\text{almacen\_disp}(j) / \text{cap\_max\_almacen})$ 
    FIN PARA

    // Calcular probabilidades
    denominador  $\leftarrow \sum_l [ \tau(i,l)^\alpha \cdot \eta(i,l)^\beta ]$ 

    PARA cada vecino  $j$ :
         $P(i \rightarrow j) \leftarrow [ \tau(i,j)^\alpha \cdot \eta(i,j)^\beta ] / \text{denominador}$ 
    FIN PARA

    // Selección por ruleta
     $r \leftarrow$  número aleatorio en  $[0, 1]$ 
    acumulado  $\leftarrow 0$ 
    PARA cada vecino  $j$  en orden:
        acumulado  $\leftarrow$  acumulado +  $P(i \rightarrow j)$ 
        SI  $r \leq$  acumulado:
            RETORNAR  $j$ 
        FIN SI
    FIN PARA
FIN

```

- FUNCIÓN: ActualizarFeromonas

ENTRADA: matriz τ , mejor solución S^* , tasa ρ , constante Q
 SALIDA: matriz τ actualizada

```

INICIO
    PARA cada arco  $(i,j)$  en  $A$ :

        SI arco  $(i,j)$  pertenece a  $S^*$ :
             $\Delta\tau(i,j) \leftarrow Q / f(S^*)$ 
        SINO:
             $\Delta\tau(i,j) \leftarrow 0$ 
        FIN SI

        // Evaporación + refuerzo
         $\tau(i,j) \leftarrow (1 - \rho) \cdot \tau(i,j) + \rho \cdot \Delta\tau(i,j)$ 

        // Aplicar límites
         $\tau(i,j) \leftarrow \max(\tau_{\min}, \min(\tau_{\max}, \tau(i,j)))$ 
    FIN PARA
FIN

```

- FUNCIÓN: Replanificar

ENTRADA: red G , envíos afectados S_{afect} , id vuelo cancelado,

```

config C
SALIDA:  PlannerResult con nuevas rutas para S_afect

INICIO
  // Penalizar arco del vuelo cancelado
   $\tau(i,j)$  del vuelo cancelado  $\leftarrow \tau_{\min}$ 

  // Reconstruir rutas solo para envíos afectados
  PARA cada envío s en S_afect:
    nueva_ruta[s]  $\leftarrow$  ConstruirRuta(s.origen, s.destino, G,  $\tau$ , C)
  FIN PARA

  RETORNAR PlannerResult(nueva_ruta)
FIN

```

3.9 Estructura de clases

A continuación se describe la arquitectura de clases diseñada para la implementación del algoritmo ACS en el componente planificador de Tasf.B2B. Las clases se organizan en cuatro capas: modelos de datos, configuración, lógica del algoritmo y salida.

3.9.1 Descripción de clases

Capa	Clase	Responsabilidad
Modelo	Airport	Representa un aeropuerto de la red. Almacena identificador, nombre, continente, capacidad máxima del almacén (500–800 maletas) y stock actual. Valida que los valores estén dentro de los rangos del negocio.
Modelo	Flight	Representa un vuelo entre dos aeropuertos. Contiene origen, destino, capacidad máxima (150–400 maletas), tiempo de traslado, frecuencia diaria, capacidad utilizada y flag de cancelación. Expone métodos para reservar capacidad y reiniciarla.
Modelo	Network	Grafo dirigido $G = (V, A)$. Gestiona aeropuertos y vuelos mediante mapas indexados por id. Expone métodos para consultar vuelos disponibles desde un nodo, verificar continente compartido y reiniciar capacidades de toda la red.
Modelo	Shipment	Representa un lote de maletas. Contiene origen, destino, cantidad de maletas, plazo máximo y estado (PENDING, IN_TRANSIT, DELIVERED, FAILED).

Modelo	<code>Route</code>	Solución construida para un Shipment. Contiene la secuencia ordenada de vuelos, tiempo total, riesgo de incumplimiento, saturación de nodos intermedios y valor de función objetivo.
Configuración	<code>ACOConfig</code>	Agrupar todos los parámetros configurables del algoritmo: α , β , ρ , Q , $\tau_{\min}$, $\tau_{\max}$, numAnts, maxIter, pesos heurísticos (w_1 , w_2 , w_3) y pesos de función objetivo (riskWeight, saturationWeight, stopsWeight). Provee un método estático <code>defaults()</code> con valores por defecto.
Algoritmo	<code>PheromoneMatrix</code>	Gestiona la matriz τ indexada por id de vuelo. Expone métodos para inicializar, evaporar, actualizar con estrategia global-best y penalizar arcos de vuelos cancelados reduciéndolos a $\tau_{\min}$.
Algoritmo	<code>AntAgent</code>	Representa una hormiga individual. Construye una ruta completa para un Shipment aplicando la regla probabilística $P(i \rightarrow j)$ usando τ y η . Calcula la función heurística compuesta con bonus por vuelo directo al destino.
Algoritmo	<code>ACSPanner</code>	Clase principal. Orquesta la colonia, gestiona el ciclo de iteraciones con semilla fija para reproducibilidad, mantiene la mejor solución global S^* y expone los métodos <code>plan()</code> y <code>replan()</code> .
Salida	<code>PlannerResult</code>	Encapsula el resultado final. Contiene el mapa de rutas por shipmentId, mejor valor de $f(S^*)$, iteraciones ejecutadas, tiempo de ejecución y métricas de cumplimiento de plazos.

3.9.2 Estructura de paquetes



3.10 Implementación en Java

El código fuente completo se encuentra disponible en el repositorio del proyecto. A continuación se presentan las tres clases centrales del algoritmo que implementan directamente la lógica ACS.

3.10.1 PheromoneMatrix.java

Gestiona la matriz $\tau(i,j)$ indexada por id de vuelo. Implementa la evaporación global, la actualización global-best y la penalización de arcos ante cancelaciones.

```
public void updateGlobalBest(List<Route> bestRoutes, double rho, double q)
{
    evaporate(rho);
    for (Route route : bestRoutes) {
        if (route.getObjectiveValue() <= 0) continue;
        double delta = q / route.getObjectiveValue();
        for (Flight f : route.getFlights()) {
            double cur = pheromones.getOrDefault(f.getId(), tauMin);
            pheromones.put(f.getId(), clamp(cur + rho * delta));
        }
    }
}

public void penalize(String flightId) {
    pheromones.put(flightId, tauMin);
}

private double clamp(double v) {
    return Math.max(tauMin, Math.min(tauMax, v));
}
```

3.10.2 AntAgent.java

Representa una hormiga individual. Implementa la regla probabilística $P(i \rightarrow j)$ y la función heurística compuesta $\eta(i,j)$ con bonus por vuelo directo al destino.

```
private Flight select(List<Flight> avail, String dest, Network network,
                    PheromoneMatrix ph, ACOConfig cfg) {
    double[] score = new double[avail.size()];
    double sum = 0;
    for (int i = 0; i < avail.size(); i++) {
        Flight f = avail.get(i);
        double tau = ph.get(f.getId());
        double eta = heuristic(f, dest, network, cfg);
        score[i] = Math.pow(tau, cfg.getAlpha()) * Math.pow(eta,
cfg.getBeta());
        sum += score[i];
    }
    if (sum <= 0) return avail.get(random.nextInt(avail.size()));
    double r = random.nextDouble() * sum, acc = 0;
    for (int i = 0; i < score.length; i++) {
        acc += score[i];
        if (r <= acc) return avail.get(i);
    }
    return avail.get(avail.size() - 1);
}

private double heuristic(Flight f, String dest, Network net, ACOConfig cfg)
{
    double eta = cfg.getW1() * (1.0 / f.getTravelTime())
        + cfg.getW2() * ((double) f.getAvailableCapacity() /
f.getCapacity())
        + cfg.getW3() * ((double)
net.getAirport(f.getDestinationId())
        .getAvailableSpace()
        /
net.getAirport(f.getDestinationId()).getWarehouseCapacity());
    if (f.getDestinationId().equals(dest)) eta *= 1.5;
    return Math.max(eta, 1e-10);
}
```

3.10.3 ACSPlanner.java

Clase principal que orquesta el ciclo ACS. Mantiene la mejor solución global S^* y expone los métodos `plan()` y `replan()`.

```
public PlannerResult plan(Network network, List<Shipment> shipments,
                        ACOConfig config) {
    pheromones = new PheromoneMatrix(network, config.getTauMin(),
                                    config.getTauMax());
    List<AntAgent> ants = buildAnts(config.getNumAnts());
    Map<String, Route> best = new HashMap<>();
    double bestF = Double.MAX_VALUE;

    for (int iter = 1; iter <= config.getMaxIterations(); iter++) {
```

```

        for (AntAgent ant : ants) {
            network.resetAllFlightCapacities();
            Map<String, Route> antR = new HashMap<>();
            double antF = 0; boolean ok = true;
            for (Shipment s : shipments) {
                Route r = ant.buildRoute(s, network, pheromones, config);
                if (r != null) { antR.put(s.getId(), r); antF +=
r.getObjectiveValue(); }
                else ok = false;
            }
            if (ok && antF < bestF) { bestF = antF; best = new
HashMap<>(antR); }
        }
        if (!best.isEmpty())
            pheromones.updateGlobalBest(new ArrayList<>(best.values()),
config.getRho(), config.getQ());
    }
    return new PlannerResult(best, bestF, config.getMaxIterations(),
System.currentTimeMillis());
}

public PlannerResult replan(Network network, List<Shipment> affected,
String cancelledId, ACOConfig config) {
    Flight c = network.getFlight(cancelledId);
    if (c != null) c.cancel();
    pheromones.penalize(cancelledId);
    return plan(network, affected, config);
}

```

3.10.4 Captura de Ejecución

```

===== RESULTADO DEL PLANIFICADOR ACS =====
Iteraciones: 200
Tiempo de ejecución: 56 ms (0.06 s)
Mejor f(S*) global: 1.426429
Envíos planificados: 5 / 5 a tiempo
-----
Route[SHP-001]: MAD -> CDG -> BOG -> LIM | time=2.00d, risk=1.0000, stops=2, f(S)=0.6750
Route[SHP-004]: BOG -> CDG -> FCO | time=1.50d, risk=0.5000, stops=1, f(S)=0.3650
Route[SHP-005]: LIM -> MAD -> CDG | time=1.50d, risk=0.5000, stops=1, f(S)=0.3864
Route[SHP-002]: CDG -> BOG | time=1.00d, risk=0.0000, stops=0, f(S)=0.0000
Route[SHP-003]: MAD -> CDG | time=0.50d, risk=0.0000, stops=0, f(S)=0.0000
=====

```

El planificador ACS ejecutó 200 iteraciones en 56 milisegundos, logrando planificar los 5 envíos dentro de sus plazos requeridos, lo que representa una tasa de cumplimiento del 100%. El valor de la función objetivo global obtenido fue de 1.426429, que refleja el costo acumulado de todas las rutas considerando tiempo de traslado y riesgo operativo.

Las rutas generadas muestran un comportamiento coherente con el modelo: los envíos con rutas directas (SHP-002 y SHP-003) obtienen riesgo cero y costo mínimo, mientras que los envíos que requieren escalas intermedias (como SHP-001 con tres tramos) presentan mayor tiempo, mayor riesgo y mayor contribución al costo total. Esto evidencia que el

algoritmo prioriza rutas directas cuando están disponibles y asigna escalas únicamente cuando la distancia o la red de aeropuertos lo exige.

4. Algoritmo 2: Búsqueda Tabú (Tabu Search, TS)

4.1 Descripción del algoritmo

La Búsqueda Tabú (Tabu Search, TS) es una metaheurística de trayectoria diseñada para abordar problemas de optimización combinatoria. A diferencia de otros métodos que se basan en procesos estocásticos o poblacionales, la TS se fundamenta en la búsqueda local enriquecida con estructuras de memoria adaptativa. Su propósito principal es permitir que el buscador explore más allá de los óptimos locales, evitando quedar atrapado en valles (o picos) de soluciones subóptimas.

El concepto central de este algoritmo es el uso de una Lista Tabú, inspirada en los procesos de la memoria humana. Esta lista actúa como un registro de los movimientos o atributos de soluciones visitadas recientemente. Al prohibir temporalmente estos movimientos (marcándolos como "tabú"), el algoritmo obliga al proceso de búsqueda a explorar regiones del espacio de soluciones que aún no han sido visitadas, evitando ciclos y redundancias.

Formalmente, la Búsqueda Tabú opera sobre un espacio de soluciones factibles. El proceso se puede resumir en los siguientes componentes clave:

1. **Generación de Vecindad:** En cada iteración, partiendo de una solución actual, se genera un conjunto de soluciones vecinas $N(s)$. Estos vecinos se obtienen aplicando una pequeña transformación o "movimiento" (por ejemplo, el intercambio de un recurso entre dos tareas o la reasignación de una ruta).
2. **Selección del Mejor Vecino:** Se evalúan todos los vecinos mediante una función objetivo $f(s)$. El algoritmo selecciona la mejor solución, incluso si esta es peor que la solución actual. Esta característica es vital, ya que es lo que permite al algoritmo "subir pendientes" para escapar de un óptimo local.
3. **Gestión de la Memoria (Lista Tabú):** Para evitar que el algoritmo regrese inmediatamente a la solución anterior y entre en un ciclo infinito, el movimiento realizado se almacena en la Lista Tabú. Esta restricción tiene una duración determinada denominada tenencia tabú (t). Una vez que transcurren t iteraciones, el movimiento vuelve a estar disponible.

4. **Criterio de Aspiración:** Es una regla de excepción. Si un movimiento está prohibido por la lista tabú, pero se descubre que aplicarlo conduce a una solución que es mejor que la mejor solución global encontrada hasta ese momento, el algoritmo ignora el estado tabú y acepta el movimiento.

4.2 Modelado del Problema de Tasf.B2B

El problema se modela como un grafo dirigido $G=(V,A)$, siguiendo la misma estructura que el algoritmo ACO para facilitar la experimentación numérica comparativa

V (Vértices): Conjunto de aeropuertos de América, Asia y Europa. Cada nodo cuenta con un atributo de capacidad de almacenamiento que oscila entre 500 y 800 maletas. El sistema debe monitorear el nivel de ocupación para evitar el colapso operativo

A (Arcos): Conjunto de vuelos entre aeropuertos. Cada arco posee los siguientes atributos:

- Capacidad de transporte: Entre 150 y 250 maletas para vuelos dentro del mismo continente, y entre 150 y 400 maletas para vuelos de diferentes continentes
- Tiempo de traslado: Definido como 0.5 -1 día para rutas continentales y 1-2 día para rutas entre distintos continentes
- Frecuencia: Los vuelos operan al menos una vez al día, con rutas seleccionadas de mayor frecuencia

En la Búsqueda Tabú, una solución "s" representa la asignación completa de todas las maletas activas en el sistema a rutas específicas que deben respetar estrictamente los plazos de entrega: máximo 1 día para el mismo continente y 2 días para continentes distintos

4.3 Función Heurística

Función objetivo

La función objetivo a minimizar combina múltiples criterios operativos para permitir una comparación justa con otros modelos:

$$f(s) = \alpha * \left(\sum_{\text{envíos}} \max(0, (\text{tiempo}_{\text{entrega}} - \text{plazo})) \right) / T_{\text{max}} + \\ \beta * \left(\sum_{\text{vuelos}} \max(0, (\text{ocupación} - \text{capacidad}_{\text{vuelo}})) \right) / C_{\text{maxvuelo}} +$$

$$\gamma * \left(\sum_{\text{aeropuertos}} \max(0, (\text{stock} - \text{capacidad}_{\text{almacén}})) \right) / C_{\text{max alm}} + \delta$$

$$* (\text{número de escalas total}) / E_{\text{max}}$$

explicación de las unidades:

$\alpha, \beta, \gamma, \delta$ son pesos puros (sin unidad) que suman 1 si quieres una media ponderada.

Incumplimiento de plazos: $\max(0, (\text{tiempo}_{\text{entrega}} - \text{plazo}))$ días

Sobrecarga de vuelos: $\max(0, (\text{ocupación} - \text{capacidad}_{\text{vuelo}}))$ número de maletas

Saturación de almacenes: $\max(0, (\text{stock} - \text{capacidad}_{\text{almacén}}))$ número de maletas

T_{max} : 2 días

$C_{\text{max vuelo}}$: 400 maletas

$C_{\text{max alm}}$: 800 maletas

E_{max} : 10 escalas

Movimientos (Vecindario)

Para generar vecinos $N(s)$, se definen tres tipos de movimientos:

- Reasignación de un lote: Este movimiento es el más sencillo: consiste en tomar un grupo de maletas (lote p) que ya tiene asignado un vuelo (v1) y cambiarlo a otro vuelo (v2) que salga hacia el mismo destino
- Intercambio de dos lotes: Aquí el algoritmo selecciona dos lotes de maletas (p1 y p2) que van en vuelos distintos y cruza sus asignaciones. El lote 1 pasa al vuelo del lote 2, y viceversa.
- Inserción de escala: Este es un movimiento más complejo. Si el planificador detecta que una ruta directa no es posible (ya sea porque no hay vuelos, están cancelados o están llenos), el algoritmo añade un aeropuerto intermedio a la trayectoria

4.4 Actualización de Feromonas y Criterio de Refuerzo

Para garantizar que la búsqueda sea eficiente y evitar el estancamiento en óptimos locales, se implementan los siguientes mecanismos de memoria y control:

Lista tabú a corto plazo: Se utiliza una cola de tamaño fijo L (con una tenencia tabú sugerida entre 5 y 15 iteraciones) que almacena los movimientos inversos recientemente realizados. Por ejemplo, si un lote de maletas fue reasignado del vuelo A al vuelo B para evitar una saturación, el movimiento inverso (de B hacia A) queda prohibido temporalmente para evitar ciclos infinitos de cálculo.

Memoria de frecuencias (Mediano/Largo plazo): Este componente registra la frecuencia con la que se utilizan ciertos vuelos o escalas en las soluciones exploradas. Si un vuelo intercontinental alcanza frecuentemente su límite de 400 maletas, el algoritmo penaliza su selección en nuevas iteraciones para diversificar la búsqueda hacia rutas alternativas menos congestionadas, cumpliendo así con el control de capacidades del requisito RAN-06.

Memoria de intensificación: Almacena la mejor solución global encontrada hasta el momento (s^*). Periódicamente, el sistema reinicia la búsqueda desde este "plan maestro" aplicando movimientos más agresivos para intentar reducir aún más el tiempo de entrega de 1 o 2 días en tramos específicos.

Criterio de aspiración: Es una regla de excepción que permite ignorar el estado "tabú" de un movimiento si este conduce a una solución que supera el récord histórico de calidad ($f(s') < f(s^*)$). Esto es vital ante una cancelación de vuelo (RAN-02), ya que permite al planificador considerar rutas que antes estaban restringidas si resultan ser la única vía para cumplir el plazo de entrega.

4.5 Adaptación para Replanificación ante Cancelaciones

Identificación: El sistema detecta automáticamente los lotes de maletas cuyo itinerario incluía el vuelo cancelado y los marca en estado "sin asignar".

Restricción Temporal (Penalización): El vuelo cancelado se ingresa inmediatamente en la Lista Tabú con una tenencia (t) alta o infinita para la sesión de replanificación, garantizando que el algoritmo no intente usar ese arco inexistente.

Reparación Local (Vecindario limitado): Para optimizar el tiempo de respuesta, TS no recalcula toda la red. En su lugar, genera un vecindario $N(s)$ compuesto únicamente por rutas alternativas para los lotes afectados, buscando vuelos con capacidad disponible (150-400 maletas) que aún permitan cumplir el plazo de entrega según el continente de destino

Ejecución rápida: Se realiza un número reducido de iteraciones (ej. 50) para obtener una solución en tiempo real.

4.6 Justificación de Selección

La elección de la Búsqueda Tabú como segundo algoritmo metaheurístico para el componente planificador se fundamenta en los siguientes pilares técnicos y operativos:

Naturaleza del problema: El traslado de equipajes de Tasf.B2B se define como un problema de enrutamiento con restricciones de capacidad (CVRP) extendido sobre una red aérea global. La literatura científica reconoce a TS como una de las metaheurísticas más efectivas para resolver problemas de optimización combinatoria de alta complejidad (NP-hard) mediante una exploración inteligente del espacio de búsqueda.

Manejo de restricciones multiobjetivo: El sistema debe gestionar simultáneamente plazos de entrega estrictos (1-2 días), capacidades de vuelo (150-400 maletas) y límites de almacenamiento (500-800 maletas). TS permite integrar estas restricciones como penalizaciones dentro de una función objetivo flexible, lo que facilita encontrar rutas factibles de bajo riesgo sin necesidad de simplificar el modelo de negocio.

Adaptabilidad dinámica y replanificación: Uno de los requisitos críticos es la replanificación ágil ante cancelaciones de vuelos (RAN-02). TS ofrece una capacidad superior para "reparar" soluciones dañadas mediante movimientos locales que excluyen el arco afectado (vuelo cancelado) a través de la lista tabú, evitando reiniciar el proceso de cálculo y manteniendo la continuidad operativa.

Control del tiempo de ejecución: Según la Exigencia 16, la simulación de periodos debe completarse en un rango de 30 a 90 minutos. Debido a que TS opera sobre una única solución y su esfuerzo computacional es directamente proporcional al número de iteraciones configuradas, permite garantizar tiempos de respuesta predecibles y ajustables para cumplir con este requisito de desempeño.

4.7 Referencias Bibliográficas

- Wassan, N. A. (2002). Tabu search variants for the mix fleet vehicle routing problem. *Journal of the Operational Research Society*, 53(7), 768–782.
<https://doi.org/10.1057/palgrave.jors.2601368> (Scopus, WoS)

- Paraskevopoulos, D. C., Repoussis, P. P., & Tarantilis, C. D. (2021). A reactive variable neighborhood tabu search for the heterogeneous fleet vehicle routing problem with time windows. *European Journal of Operational Research*, 288(1), 129–140. <https://doi.org/10.1016/j.ejor.2020.05.050> (Scopus)
- Brandão, J. (2020). A tabu search algorithm for the heterogeneous fixed fleet vehicle routing problem. *Computers & Operations Research*, *117*, Article 104881. <https://doi.org/10.1016/j.cor.2020.104881> (Scopus)
- Kyriakakis, N. A., Sevastopoulos, I., Marinaki, M., & Marinakis, Y. (2022). A hybrid Tabu search – Variable neighborhood descent algorithm for the cumulative capacitated vehicle routing problem with time windows in humanitarian applications. *Computers & Industrial Engineering*, *164*, Article 107868. <https://doi.org/10.1016/j.cie.2021.107868> (Scopus)
- Glover, F. (1989). Tabu search—Part I. *ORSA Journal on Computing*, 1(3), 190–206. <https://doi.org/10.1287/ijoc.1.3.190> (Scopus)
- Ruf, C., Schiffels, S., Kolisch, R., & Frey, M. M. (2022). A data-driven approach for baggage handling operations at airports. *Transportation Science*, 56(5), 1125–1440. <https://doi.org/10.1287/trsc.2022.1127> (Scopus)

4.8 Pseudocódigo

A continuación se presenta el pseudocódigo formal del algoritmo de Tabu-search adaptado a la situación de Tasf.B2B

- Función para construir solución inicial

```
función construirSolucionInicial(envios, grafo):
    solucion ← Solucion vacía

    para cada envio en envios:
        ruta ← construirRuta(envio, grafo)
        solucion.agregarRuta(ruta)
    retornar solucion
```

```
función construirRuta(envio, grafo):
```

```

ruta ← Ruta(envio)
// FASE 1: secuencia de aeropuertos (BFS precomputado)
secuencia ← grafo.getRutaCorta(envio.origen, envio.destino)
// Ejemplo: ["EBCI", "SKBO", "SPIM"]
si secuencia está vacía:
    ruta.sinSolucion ← true
    retornar ruta
// FASE 2: asignar el primer vuelo disponible en cada tramo
tiempoActual ← envio.minutosRegistro

para i desde 0 hasta len(secuencia) - 2:
    desde ← secuencia[i]
    hacia ← secuencia[i+1]

    vuelo ← grafo.primerVueloDisponible(desde, hacia, tiempoActual)
    // "primer vuelo que sale ≥ tiempoActual + 30 min de margen"
    si vuelo == null:
        ruta.sinSolucion ← true
        retornar ruta

    ruta.agregarVuelo(vuelo)

    salidaAbsoluta ← proximaSalidaAbsoluta(tiempoActual,
vuelo.salidaMin, margen=30)
    tiempoActual ← salidaAbsoluta + vuelo.duracionMinutos
    retornar ruta

- Función para obtener costo total:
función getCostoTotal(solucion):
    si solucion.dirty == false:
        retornar solucion.costoTotal

suma ← 0
para cada ruta en solucion.asignaciones:
    si ruta.sinSolucion:
        suma ← suma + PENALIZACION (100.000 min)
    sino:
        suma ← suma + ruta.calcularTiempoTotal()

solucion.costoTotal ← suma

```

```

    solucion.dirty ← false
    retornar suma
- ALGORITMO PRINCIPAL:
función ejecutar(solucionInicial, envios):
    actual ← clonar(solucionInicial)
    mejorGlobal ← clonar(solucionInicial)
    listaTabu ← Cola FIFO vacía // máx. 'tenencia' elementos

    para iter desde 1 hasta maxIteraciones:
        // a) Generar vecinos candidatos
        candidatos ← generarCandidatos(actual, envios, tamanoMuestra)
        si candidatos está vacío: terminar

        // b) Elegir el mejor movimiento permitido
        elegido ← null
        mejorDelta ← +∞

        para cada mov en candidatos:
            esTabu ← listaTabu.contiene(mov.claveTabu)
            costoNuevo ← getCostoTotal(actual) + mov.deltaCosto
            superaAspiracion ← costoNuevo < getCostoTotal(mejorGlobal)

            // Aceptar si: no es tabú O supera el mejor global (criterio
de aspiración)
            si (NO esTabu) O superaAspiracion:
                si mov.deltaCosto < mejorDelta:
                    mejorDelta ← mov.deltaCosto
                    elegido ← mov

        // Si todos son tabú y ninguno supera aspiración → forzar el menos
malo

        si elegido == null:
            elegido ← candidatos.minPor(deltaCosto)

        si elegido == null: continuar

        // c) Aplicar el movimiento (modifica 'actual' en el lugar)
        actual.agregarRuta(elegido.rutaNueva) // sobrescribe la ruta
anterior del envío

```

```

    actual.marcarDirty()

    // d) Registrar en lista tabú
    listaTabu.agregarAlFinal(elegido.claveTabu)
    si listaTabu.tamaño > tenencia:
        listaTabu.eliminarElPrimero()          // expira el movimiento
más antiguo

    // e) Actualizar mejor global
    si getCostoTotal(actual) < getCostoTotal(mejorGlobal):
        mejorGlobal ← clonar(actual)

    retornar mejorGlobal

```

- Función para generar candidatos

```

función generarCandidatos(solucion, todosEnvios, tamanoMuestra):
    candidatos ← Lista vacía
    muestra ← seleccionarAleatoriosSinReemplazo(todosEnvios,
tamanoMuestra)
    para cada envio en muestra:
        rutaActual ← solucion.getRuta(envio)
        si rutaActual == null: continuar
        // Tipo 1: cambiar el primer vuelo por otro en el mismo tramo
        mov1 ← generarProximoVuelo(envio, rutaActual)
        si mov1 != null: candidatos.agregar(mov1)

        // Tipo 2: introducir o cambiar la escala intermedia
        mov2 ← generarHubAlternativo(envio, rutaActual)
        si mov2 != null: candidatos.agregar(mov2)
    retornar candidatos

```

- Función para generar próximo vuelo

```

función generarProximoVuelo(envio, rutaActual):
    si rutaActual.sinSolucion o rutaActual.vuelos está vacía: retornar null
    primerVuelo ← rutaActual.vuelos[0]
    desde ← primerVuelo.origen
    hacia ← primerVuelo.destino
    // Buscar otro vuelo en el mismo tramo con salida diferente
    siguienteVuelo ← null
    mejorSalida ← ∞

```

```

    para cada vuelo en grafo.getVuelosDesde(desde):
        si vuelo.destino != hacia: continuar
        si vuelo.salidaMin == primerVuelo.salidaMin: continuar // mismo
vuelo
        salidaAbs ← proximaSalidaAbsoluta(envio.minutosRegistro,
vuelo.salidaMin, 30)
        si salidaAbs < mejorSalida:
            mejorSalida ← salidaAbs
            siguienteVuelo ← vuelo

    si siguienteVuelo == null: retornar null
    // Reconstruir ruta con el nuevo primer vuelo, reasignando tramos
siguientes
    rutaNueva ← reconstruirRutaConPrimerVuelo(envio, rutaActual,
siguienteVuelo)
    si rutaNueva == null: retornar null

    retornar Movimiento(envio.clave, rutaActual, rutaNueva)

```

- **Función para generar Hub alternativo:**

```

función generarHubAlternativo(envio, rutaActual):
    origen ← envio.origen
    destino ← envio.destino

    // Hubs que ya usa la ruta actual (para no repetirlos)
    hubsActuales ← {vuelos[i].destino para i en 0..len-2}
    // Candidatos: X tal que existen vuelos ORIG→X y X→DEST
    hubsValidos ← []
    para cada X en destinos_alcanzables_desde(origen):
        si X == destino o X == origen: continuar
        si X en hubsActuales: continuar
        si existe vuelo X→destino:
            hubsValidos.agregar(X)

    si hubsValidos está vacío: retornar null

    hub ← elegirAleatorio(hubsValidos)

    // Construir ruta ORIG → hub → DEST
    tiempoActual ← envio.minutosRegistro

```



```

v1 ← grafo.primerVueloDisponible(origen, hub, tiempoActual)
si v1 == null: retornar null

llegada1 ← proximaSalidaAbsoluta(tiempoActual, v1.salidaMin, 30) +
v1.duracion
v2 ← grafo.primerVueloDisponible(hub, destino, llegada1)
si v2 == null: retornar null

rutaNueva ← Ruta(envio, [v1, v2])
retornar Movimiento(envio.clave, rutaActual, rutaNueva)

```

4.9 Estructura de clases

A continuación se describe la arquitectura de clases diseñada para la implementación del algoritmo Tabu-search en el componente planificador de Tasf.B2B.

4.9.1 Descripción de clases

Capa	Clase	Responsabilidad
Modelo	Aeropuerto	Representa un nodo de la red de transporte. Almacena código, ciudad, país, huso horario (GMT) y capacidad máxima de almacenamiento. Controla la ocupación dinámica mediante agregarMaletas() y liberarMaletas(), validando que no se exceda la capacidad.
Modelo	Vuelo	Representa un arco del grafo entre dos aeropuertos. Contiene origen, destino, hora de salida y llegada (en minutos desde medianoche), capacidad máxima y ocupación actual. Calcula duración (con soporte para vuelos nocturnos), tiempo de espera para el transbordo y gestiona la reserva de capacidad.
Modelo	Envio	Representa un lote de maletas a transportar. Contiene id, aeropuerto origen (inferido del archivo de datos), destino, fecha-hora de registro, cantidad de maletas e id del cliente. Expone getMinutosRegistro() como punto de inicio temporal para la construcción de rutas.

Modelo	Ruta	Solución construida para un Envío. Contiene la secuencia ordenada de vuelos desde origen hasta destino. Calcula el tiempo total de entrega (vuelos + esperas en escalas) mediante calcularTiempoTotal(). Incluye sinSolucion para marcar envíos sin camino viable. Soporta clonación para ser copiada sin afectar a la solución original.
Algoritmo	GrafoVuelos	Grafo dirigido $G = (V, A)$. Indexa vuelos por aeropuerto origen para consulta en $O(1)$. Precomputa con BFS la ruta de menor número de escalas entre todos los pares de aeropuertos (≈ 870 pares) una sola vez al construirse. Expone getRutaCorta(), getVuelosDesde() y primerVueloDisponible().
Algoritmo	SolucionInicial	Construye la solución de partida para el Tabu Search en dos fases por envío: (1) obtiene la secuencia de aeropuertos con menor número de escalas desde GrafoVuelos (BFS precomputado), y (2) asigna el primer vuelo disponible en cada tramo con una estrategia greedy por tiempo de salida, respetando 30 min de margen de transbordo.
Algoritmo	Solucion	Estado completo del sistema: mapa de idEnvío $\rightarrow$ Ruta. Implementa la función objetivo getCostoTotal() (suma de tiempos de entrega + penalización de 100.000 min por envío sin ruta), con caché invalidada por marcarDirty(). Soporta clonación profunda para explorar vecinos sin modificar la solución actual.
Algoritmo	Movimiento	Representa un cambio atómico en el espacio de soluciones: reasignar la ruta de un envío específico. Encapsula la ruta anterior, la ruta nueva, el deltaCosto (negativo = mejora) y la claveTabu (firma "idEnvío:ORIG-HUB-DEST" de la ruta anterior), que es la entrada que se registra en la lista tabú para evitar deshacer el cambio.
Algoritmo	Vecindad	Genera los movimientos candidatos para cada iteración. Para una muestra aleatoria de envíos evalúa dos tipos de movimiento: Tipo 1 (próximo vuelo en el primer tramo, mismo par origen-destino pero salida diferente) y Tipo 2 (hub alternativo: introduce o cambia la escala intermedia). El muestreo aleatorio permite escalar a 330K+ envíos sin evaluar la vecindad completa.
Algoritmo	TabuSearch	Núcleo del algoritmo metaheurístico. En cada iteración: genera candidatos con Vecindad, selecciona el mejor movimiento no tabú (o tabú que supere la mejor solución global por criterio de aspiración), lo aplica a la solución actual, actualiza la lista tabú (cola FIFO de tenencia fija) y preserva la mejor solución global.

		Parámetros configurables: maxIteraciones, tenenciaTabu y tamanoMuestra.
Datos	DataLoader	Carga y parsea los archivos de entrada: vuelos (vuelos.txt), aeropuertos y envíos (uno por aeropuerto origen). Construye las instancias de modelo y las entrega listas para inicializar el GrafoVuelos.

4.9.2 Estructura de paquetes



4.10 Implementación en Java

El código fuente completo se encuentra disponible en el repositorio del proyecto. A continuación se presentan las clases centrales del algoritmo que implementan la lógica de tabú search.

4.10.1 TabuSearch.java

Contiene el loop de los 5 pasos del algoritmo tabú: generación de candidatos (vecindad), selección del mejor movimiento, aplicación del movimiento, actualización de la lista tabu y actualización del mejor global

```
public class TabuSearch {
    private final int maxIteraciones;
    private final int tenenciaTabu;
    private final int tamanoMuestra;
    private final GrafoVuelos grafo;
```

```

private final Vecindad    vecindad;
private final LinkedList<String> listaTabu;

public TabuSearch(GrafoVuelos grafo, int maxIteraciones, int
tenenciaTabu, int tamanoMuestra) {
    this.grafo          = grafo;
    this.maxIteraciones = maxIteraciones;
    this.tenenciaTabu   = tenenciaTabu;
    this.tamanoMuestra  = tamanoMuestra;
    this.vecindad       = new Vecindad(grafo);
    this.listaTabu      = new LinkedList<>();
}

public TabuSearch(GrafoVuelos grafo) {
    this(grafo, 200, 30, 200);
}

public Solucion ejecutar(Solucion solucionInicial, List<Envio> envios)
{
    Solucion actual          = solucionInicial.clonar();
    Solucion mejorGlobal     = solucionInicial.clonar();

    System.out.println("\n[TabuSearch] Iniciando...");
    System.out.printf("[TabuSearch] Costo inicial: %.0f | Iter: %d |
Muestra: %d | Tenencia: %d\n",
        actual.getCostoTotal(), maxIteraciones, tamanoMuestra,
tenenciaTabu);

    for (int iter = 1; iter <= maxIteraciones; iter++) {

        List<Movimiento> candidatos =
vecindad.generarCandidatos(actual, envios, tamanoMuestra);

        if (candidatos.isEmpty()) {
            System.out.println("[TabuSearch] Sin candidatos en iter " +
iter + ". Terminando.");
            break;
        }
        Movimiento elegido          = null;
        double mejorDelta           = Double.MAX_VALUE;

        for (Movimiento mov : candidatos) {
            boolean esTabu          =
listaTabu.contains(mov.getClaveTabu());
            double costoNuevo       = actual.getCostoTotal() +
mov.getDeltaCosto();

            // Criterio de aspiración: tabú pero mejor que el mejor
global
            boolean superaAspiracion = costoNuevo <
mejorGlobal.getCostoTotal();

            if (!esTabu || superaAspiracion) {
                if (mov.getDeltaCosto() < mejorDelta) {
                    mejorDelta = mov.getDeltaCosto();
                    elegido     = mov;
                }
            }
        }
    }
}

```

```

        if (elegido == null) {
            // Todos los candidatos están tabú y ninguno supera la
aspiración.
            // Forzamos el menos malo de todos (sin restricción tabú)
para no estancarnos.
            elegido = candidatos.stream()
                .min((a, b) -> Double.compare(a.getDeltaCosto(),
b.getDeltaCosto()))
                .orElse(null);
        }

        if (elegido == null) continue;
        aplicarMovimiento(actual, elegido);
        listaTabu.addLast(elegido.getClaveTabu());
        if (listaTabu.size() > tenenciaTabu) {
            listaTabu.removeFirst(); // expira el movimiento más
antiguo
        }
        if (actual.getCostoTotal() < mejorGlobal.getCostoTotal()) {
            mejorGlobal = actual.clonar();
        }
        if (iter % 10 == 0 || iter == 1) {
            System.out.printf(
                "[TabuSearch] Iter %3d | CostoActual: %9.0f |
MejorGlobal: %9.0f | Tabú: %d/%d | Delta: %+.0f%n",
                iter,
                actual.getCostoTotal(),
                mejorGlobal.getCostoTotal(),
                listaTabu.size(),
                tenenciaTabu,
                elegido.getDeltaCosto()
            );
        }
    }

    System.out.printf("%n[TabuSearch] Finalizado.%n");
    System.out.printf("[TabuSearch] Costo inicial : %.0f min%n",
solucionInicial.getCostoTotal());
    System.out.printf("[TabuSearch] Mejor global : %.0f min%n",
mejorGlobal.getCostoTotal());
    System.out.printf("[TabuSearch] Mejora total : %.0f min
(%.2f%%)%n",
        solucionInicial.getCostoTotal() - mejorGlobal.getCostoTotal(),
        (1.0 - mejorGlobal.getCostoTotal() /
solucionInicial.getCostoTotal()) * 100
    );

    return mejorGlobal;
}

private void aplicarMovimiento(Solucion solucion, Movimiento
movimiento) {
    Ruta rutaNueva = movimiento.getRutaNueva();
    solucion.agregarRuta(rutaNueva); // sobrescribe la ruta anterior
del mismo envío
    solucion.marcarDirty();
}
}

```

4.10.2 Vecindad.java

Define los 2 tipos de movimientos que explora el algoritmo: el tipo 1 es el mismo tramo, vuelo con salida alternativa y el tipo 2, ruta diferente con aeropuerto intermedio diferente

```
public class Vecindad {

    private final GrafoVuelos grafo;
    private final Random rng;

    private List<String> hubsPrincipales;

    public Vecindad(GrafoVuelos grafo, long semilla) {
        this.grafo = grafo;
        this.rng = new Random(semilla);
    }

    public Vecindad(GrafoVuelos grafo) {
        this(grafo, 42L);
    }

    public List<Movimiento> generarCandidatos(Solucion solucion,
                                              List<Envio> todosEnvios,
                                              int tamanoMuestra) {
        List<Movimiento> candidatos = new ArrayList<>();
        List<Envio> muestra = muestrar(todosEnvios, tamanoMuestra);

        for (Envio envio : muestra) {
            Ruta rutaActual = solucion.getRuta(envio);
            if (rutaActual == null) continue;
            String claveEnvio = envio.getOrigen() + "-" + envio.getId();
            Movimiento mov1 = generarProximoVuelo(envio, rutaActual,
claveEnvio);
            if (mov1 != null) candidatos.add(mov1);
            Movimiento mov2 = generarHubAlternativo(envio, rutaActual,
claveEnvio);
            if (mov2 != null) candidatos.add(mov2);
        }

        return candidatos;
    }

    private Movimiento generarProximoVuelo(Envio envio, Ruta rutaActual,
String claveEnvio) {
        if (rutaActual.isSinSolucion() || rutaActual.getVuelos().isEmpty())
return null;

        Vuelo primerVuelo = rutaActual.getVuelos().get(0);
        String desde = primerVuelo.getOrigen();
        String hacia = primerVuelo.getDestino();

        Vuelo siguiente = siguienteVueloEnTramo(
            desde, hacia,
            primerVuelo.getSalidaMinutos(),
            envio.getMinutosRegistro()
        );

        if (siguiente == null || siguiente == primerVuelo) return null;
    }
}
```

```

        Ruta rutaNueva = construirRutaConPrimerVuelo(envio, rutaActual,
siguiente);
        if (rutaNueva == null) return null;

        return new Movimiento(claveEnvio, rutaActual, rutaNueva);
    }

    private Vuelo siguienteVueloEnTramo(String desde, String hacia,
                                        int salidaActualMin, int
tiempoRegistroMin) {
        List<Vuelo> candidatos = grafo.getVuelosDesde(desde);
        Vuelo mejor = null;
        int mejorSalida = Integer.MAX_VALUE;

        for (Vuelo v : candidatos) {
            if (!v.getDestino().equals(hacia)) continue;
            if (v.getSalidaMinutos() == salidaActualMin) continue;
            int salidaAbs = GrafoVuelos.proximaSalidaAbsoluta(
                tiempoRegistroMin, v.getSalidaMinutos(), 30
            );
            if (salidaAbs < mejorSalida) {
                mejorSalida = salidaAbs;
                mejor = v;
            }
        }
        return mejor;
    }

    private Ruta construirRutaConPrimerVuelo(Envio envio, Ruta original,
Vuelo nuevoVuelo) {
        Ruta nueva = new Ruta(envio);
        nueva.agregarVuelo(nuevoVuelo);

        int salidaAbs = GrafoVuelos.proximaSalidaAbsoluta(
            envio.getMinutosRegistro(), nuevoVuelo.getSalidaMinutos(), 30
        );
        int tiempoActual = salidaAbs + nuevoVuelo.getDuracionMinutos();

        List<Vuelo> vuelosOriginales = original.getVuelos();
        for (int i = 1; i < vuelosOriginales.size(); i++) {
            Vuelo segmento = vuelosOriginales.get(i);
            Vuelo reemplazo = grafo.primerVueloDisponible(
                segmento.getOrigen(), segmento.getDestino(), tiempoActual
            );
            if (reemplazo == null) return null;
            nueva.agregarVuelo(reemplazo);
            int sal = GrafoVuelos.proximaSalidaAbsoluta(
                tiempoActual, reemplazo.getSalidaMinutos(), 30
            );
            tiempoActual = sal + reemplazo.getDuracionMinutos();
        }
        return nueva;
    }

    private Movimiento generarHubAlternativo(Envio envio, Ruta rutaActual,
String claveEnvio) {
        String origen = envio.getOrigen();
        String destino = envio.getDestino();

        Set<String> hubsActuales = new HashSet<>();
        List<Vuelo> vuelos = rutaActual.getVuelos();

```

```

        for (int i = 0; i < vuelos.size() - 1; i++) {
            hubsActuales.add(vuelos.get(i).getDestino());
        }

        List<String> hubs = hubsValidos(origen, destino, hubsActuales);
        if (hubs.isEmpty()) return null;

        String hub = hubs.get(rng.nextInt(hubs.size()));
        int tiempoActual = envio.getMinutosRegistro();

        Vuelo v1 = grafo.primerVueloDisponible(origen, hub, tiempoActual);
        if (v1 == null) return null;

        int sal1 = GrafoVuelos.proximaSalidaAbsoluta(tiempoActual,
v1.getSalidaMinutos(), 30);
        int llegada1 = sal1 + v1.getDuracionMinutos();

        Vuelo v2 = grafo.primerVueloDisponible(hub, destino, llegada1);
        if (v2 == null) return null;

        Ruta rutaNueva = new Ruta(envio);
        rutaNueva.agregarVuelo(v1);
        rutaNueva.agregarVuelo(v2);

        return new Movimiento(claveEnvio, rutaActual, rutaNueva);
    }

    private List<String> hubsValidos(String origen, String destino,
Set<String> excluir) {
        Set<String> desdeOrigen = new HashSet<>();
        for (Vuelo v : grafo.getVuelosDesde(origen)) {
            desdeOrigen.add(v.getDestino());
        }

        List<String> hubs = new ArrayList<>();
        for (String candidato : desdeOrigen) {
            if (candidato.equals(destino)) continue;
            if (candidato.equals(origen)) continue;
            if (excluir.contains(candidato)) continue;

            boolean tieneConexion =
grafo.getVuelosDesde(candidato).stream()
                .anyMatch(v -> v.getDestino().equals(destino));
            if (tieneConexion) hubs.add(candidato);
        }
        return hubs;
    }

    private List<Envio> muestrear(List<Envio> lista, int n) {
        if (n >= lista.size()) return new ArrayList<>(lista);
        List<Envio> copia = new ArrayList<>(lista);
        Collections.shuffle(copia, rng);
        return copia.subList(0, n);
    }
}

```


4.10.3 Movimiento.java

Define los 2 tipos de movimientos que explora el algoritmo: el tipo 1 es el mismo tramo, vuelo con salida alternativa y el tipo 2, ruta diferente con aeropuerto intermedio diferente

```
public class Movimiento {

    private final String claveEnvio;
    private final Ruta    rutaAnterior;
    private final Ruta    rutaNueva;
    private final double  deltaCosto;
    private final String  claveTabu;

    public Movimiento(String claveEnvio, Ruta rutaAnterior, Ruta rutaNueva)
    {
        this.claveEnvio    = claveEnvio;
        this.rutaAnterior  = rutaAnterior;
        this.rutaNueva     = rutaNueva;
        this.deltaCosto    = calcularDelta();
        this.claveTabu     = claveEnvio + ":" + firmaRuta(rutaAnterior);
    }

    private double calcularDelta() {
        int costoAnterior = rutaAnterior.isSinSolucion()
            ? 100_000 : rutaAnterior.calcularTiempoTotal();
        int costoNuevo    = rutaNueva.isSinSolucion()
            ? 100_000 : rutaNueva.calcularTiempoTotal();
        return costoNuevo - costoAnterior;
    }

    public static String firmaRuta(Ruta ruta) {
        if (ruta == null || ruta.isSinSolucion() ||
            ruta.getVuelos().isEmpty()) {
            return "SIN_RUTA";
        }
        String first = ruta.getVuelos().get(0).getOrigen();
        String hops  = ruta.getVuelos().stream()
            .map(Vuelo::getDestino)
            .collect(Collectors.joining("-"));
        return first + "-" + hops;
    }

    public String  getClaveEnvio()    { return claveEnvio; }
    public Ruta    getRutaAnterior() { return rutaAnterior; }
    public Ruta    getRutaNueva()    { return rutaNueva; }
    public double  getDeltaCosto()    { return deltaCosto; }
    public String  getClaveTabu()     { return claveTabu; }

    @Override
    public String toString() {
        return String.format("Mov[%s] %s -> %s (delta=%.0f)",
            claveEnvio,
            firmaRuta(rutaAnterior),
            firmaRuta(rutaNueva),
            deltaCosto);
    }
}
```

4.10.4 SolucionInicial.java

Genera la solución de arranque en 2 fases: BFS para la secuencia de aeropuertos + greedy para asignar vuelos concretos por tiempo mínimo

```
public class SolucionInicial {
    private final GrafoVuelos grafo;
    public SolucionInicial(GrafoVuelos grafo) {
        this.grafo = grafo;
    }

    public Solucion construir(List<Envio> envios) {
        Solucion solucion = new Solucion();
        int procesados = 0;
        int sinRuta = 0;

        for (Envio envio : envios) {
            Ruta ruta = construirRuta(envio);
            solucion.agregarRuta(ruta);

            if (ruta.isSinSolucion()) sinRuta++;

            procesados++;
            if (procesados % 10_000 == 0) {
                System.out.printf("[SolucionInicial] %d/%d envíos
procesados (sin ruta: %d)%n",
                                procesados, envios.size(), sinRuta);
            }
        }

        System.out.printf("[SolucionInicial] Completado: %d envíos, %d sin
ruta%n",
                            procesados, sinRuta);
        return solucion;
    }

    public Ruta construirRuta(Envio envio) {
        Ruta ruta = new Ruta(envio);

        if (envio.getOrigen().equals(envio.getDestino())) {
            return ruta;
        }

        List<String> secuencia = grafo.getRutaCorta(envio.getOrigen(),
envio.getDestino());

        if (secuencia.isEmpty()) {
            ruta.setSinSolucion(true);
            return ruta;
        }

        int tiempoActual = envio.getMinutosRegistro();

        for (int i = 0; i < secuencia.size() - 1; i++) {
            String desde = secuencia.get(i);
            String hacia = secuencia.get(i + 1);

            Vuelo vuelo = grafo.primerVueloDisponible(desde, hacia,
tiempoActual);
```

```

        if (vuelo == null) {
            ruta.setSinSolucion(true);
            return ruta;
        }

        ruta.agregarVuelo(vuelo);

        int salidaAbsoluta = GrafoVuelos.proximaSalidaAbsoluta(
            tiempoActual, vuelo.getSalidaMinutos(), 30);
        tiempoActual = salidaAbsoluta + vuelo.getDuracionMinutos();
    }
    return ruta;
}
}

```

4.10.5 Solucion.java

Representa el estado completo. La función objetivo suma los tiempos de entrega y aplica penalización de 100,000 min a envíos sin ruta viable.

```

public class Solucion {

    private static final int PENALIZACION_SIN_RUTA = 100_000; // minutos
    equivalentes
    private final Map<String, Ruta> asignaciones;
    private double costoTotal;
    private boolean dirty;
    public Solucion() {
        this.asignaciones = new LinkedHashMap<>();
        this.costoTotal = Double.MAX_VALUE;
        this.dirty = true;
    }

    private Solucion(Map<String, Ruta> asignaciones) {
        this.asignaciones = new LinkedHashMap<>(asignaciones);
        this.dirty = true;
    }

    public void agregarRuta(Ruta ruta) {
        asignaciones.put(claveGlobal(ruta.getEnvio()), ruta);
        dirty = true;
    }

    public Ruta getRuta(Envio envio) {
        return asignaciones.get(claveGlobal(envio));
    }

    private static String claveGlobal(Envio envio) {
        return envio.getOrigen() + "-" + envio.getId();
    }

    public Collection<Ruta> getRutas() {
        return asignaciones.values();
    }

    public int getTotalEnvios() {
        return asignaciones.size();
    }
}

```

```

    public double getCostoTotal() {
        if (dirty) recalcularCosto();
        return costoTotal;
    }

    private void recalcularCosto() {
        long suma = 0;
        int sinRuta = 0;

        for (Ruta ruta : asignaciones.values()) {
            if (ruta.isSinSolucion()) {
                sinRuta++;
            } else {
                int t = ruta.calcularTiempoTotal();
                suma += (t == Integer.MAX_VALUE) ? PENALIZACION_SIN_RUTA :
t;
            }
        }

        costoTotal = suma + (long) sinRuta * PENALIZACION_SIN_RUTA;
        dirty = false;
    }

    public void marcarDirty() {
        dirty = true;
    }

    public int contarEnviosSinRuta() {
        return (int)
asignaciones.values().stream().filter(Ruta::isSinSolucion).count();
    }

    public int contarEnviosConRuta() {
        return getTotalEnvios() - contarEnviosSinRuta();
    }

    public double getTiempoPromedioEntrega() {
        return asignaciones.values().stream()
            .filter(r -> !r.isSinSolucion())
            .mapToInt(Ruta::calcularTiempoTotal)
            .filter(t -> t != Integer.MAX_VALUE)
            .average()
            .orElse(0.0);
    }

    public Solucion clonar() {
        Map<String, Ruta> copiaAsignaciones = new LinkedHashMap<>();
        for (Map.Entry<String, Ruta> entrada : asignaciones.entrySet()) {
            copiaAsignaciones.put(entrada.getKey(), new
Ruta(entrada.getValue()));
        }
        return new Solucion(copiaAsignaciones);
    }

    @Override
    public String toString() {
        return String.format(
            "Solucion { envios=%d, conRuta=%d, sinRuta=%d, costoTotal=%.0f,
promedio=%.1f min }",
            getTotalEnvios(),
            contarEnviosConRuta(),

```

```

        contarEnviosSinRuta(),
        getCostoTotal(),
        getTiempoPromedioEntrega()
    );
}

public void imprimirResumen() {
    System.out.println("=== RESUMEN DE SOLUCIÓN ===");
    System.out.printf("  Envíos totales      : %d%n", getTotalEnvios());
    System.out.printf("  Con ruta          : %d%n",
        contarEnviosConRuta());
    System.out.printf("  Sin ruta viable   : %d%n",
        contarEnviosSinRuta());
    System.out.printf("  Costo total       : %.0f min%n",
        getCostoTotal());
    System.out.printf("  Tiempo promedio  : %.1f min%n",
        getTiempoPromedioEntrega());
}
}

```

4.10.6 Captura de Ejecución

Resultados de solución inicial (greedy):

```

=== SOLUCIÓN INICIAL ===
[SolucionInicial] Completado: 1500 envíos, 0 sin ruta
=== RESUMEN DE SOLUCIÓN ===
Envíos totales      : 1500
Con ruta           : 1500
Sin ruta viable     : 0
Costo total         : 1145180 min
Tiempo promedio    : 763.5 min

```

Evolución del tabu-search: Se ve que el costo baja en cada iteración ya que el delta siempre es negativo, lo que indica que el algoritmo nunca se estancó en las 100 iteraciones ejecutadas. Esto sugiere que con más iteraciones podría seguir mejorando.

```
[TabuSearch] Iniciando...
[TabuSearch] Costo inicial: 1145180 | Iter: 100 | Muestra: 200 | Tenencia: 30
[TabuSearch] Iter 1 | CostoActual: 1143568 | MejorGlobal: 1143568 | Tabú: 1/30 | Delta: -1612
[TabuSearch] Iter 10 | CostoActual: 1134517 | MejorGlobal: 1134517 | Tabú: 10/30 | Delta: -932
[TabuSearch] Iter 20 | CostoActual: 1124005 | MejorGlobal: 1124005 | Tabú: 20/30 | Delta: -1139
[TabuSearch] Iter 30 | CostoActual: 1113421 | MejorGlobal: 1113421 | Tabú: 30/30 | Delta: -611
[TabuSearch] Iter 40 | CostoActual: 1103104 | MejorGlobal: 1103104 | Tabú: 30/30 | Delta: -1334
[TabuSearch] Iter 50 | CostoActual: 1092584 | MejorGlobal: 1092584 | Tabú: 30/30 | Delta: -1029
[TabuSearch] Iter 60 | CostoActual: 1081670 | MejorGlobal: 1081670 | Tabú: 30/30 | Delta: -1110
[TabuSearch] Iter 70 | CostoActual: 1072295 | MejorGlobal: 1072295 | Tabú: 30/30 | Delta: -949
[TabuSearch] Iter 80 | CostoActual: 1062128 | MejorGlobal: 1062128 | Tabú: 30/30 | Delta: -1205
[TabuSearch] Iter 90 | CostoActual: 1052917 | MejorGlobal: 1052917 | Tabú: 30/30 | Delta: -969
[TabuSearch] Iter 100 | CostoActual: 1042814 | MejorGlobal: 1042814 | Tabú: 30/30 | Delta: -841

[TabuSearch] Finalizado.
[TabuSearch] Costo inicial : 1145180 min
[TabuSearch] Mejor global : 1042814 min
[TabuSearch] Mejora total : 102366 min (8.94%)

=== SOLUCIÓN FINAL ===
=== RESUMEN DE SOLUCIÓN ===
Envíos totales : 1500
Con ruta : 1500
Sin ruta viable : 0
Costo total : 1042814 min
Tiempo promedio : 695.2 min
```